Faculty of Natural, Mathematical, and Engineering Sciences
Department of Informatics

Software Engineering Group Project

KCL Ticketing System

PackItUpOnly
Amey Tripathi
Hafsa Bhudye
Pasquale Benjamin Fuccio
Jasmeen Sethi
Author Five
Author Six
Author Seven
Author Eight

March 30, 2026

Contents

1 Introduction **1**

2 Product **2**
2.1 Objectives, context and key stakeholders . . . 2
2.1.1 Project Objectives . . . 3
2.1.2 Key Stakeholders . . . 3
2.1.3 Value Proposition . . . 3
2.2 Requirements and specifications . . . 4

3 Design and implementation **7**
3.1 System Architecture . . . 7
3.1.1 System Layers . . . 7
3.2 Component Design . . . 7
3.2.1 Frontend Pages . . . 8
3.2.2 Administrative Pages . . . 8
3.2.3 Shared Components . . . 8
3.2.4 State Management and API Communication . . . 8
3.3 Backend Structure . . . 8
3.3.1 Django Applications . . . 9
3.3.2 View Organisation . . . 9
3.3.3 Permissions and Access Control . . . 9
3.3.4 AI Helper Development with Gemini API . . . 9
3.4 Database Design . . . 10
3.5 API Design . . . 11
3.5.1 Authentication Endpoints . . . 11
3.5.2 Ticket Management Endpoints . . . 11
3.5.3 Meeting and Staff Endpoints . . . 12
3.5.4 Administrative Endpoints . . . 12
3.6 System Workflows . . . 12
3.7 Design decisions and rationale . . . 14
3.7.1 Separation of frontend and backend (React and Django REST) . . . 14
3.7.2 Generative AI: Google Gemini API versus self-hosted inference (e.g. Ollama) . . . 15
3.7.3 Authentication and data store choices . . . 15
3.7.4 How decisions are reflected in the end product . . . 16
3.7.5 Benefits of the chosen architecture . . . 16
3.7.6 Limitations and trade-offs . . . 16

3.7.7 Other alternative approaches considered . . . 16

4 Testing **17**
4.1 Testing approach . . . 17
4.2 Automated testing . . . 17
4.2.1 Suite scope and structure . . . 17
4.2.2 Backend testing (unit and integration) . . . 18
4.2.3 Frontend testing (component and behaviour) . . . 18
4.2.4 Execution pipeline . . . 19
4.2.5 Current observed run status . . . 19
4.3 Manual testing . . . 20
4.3.1 Purpose of manual testing . . . 20
4.3.2 Extent of reliance on manual testing . . . 20
4.3.3 Limits of automated and manual testing . . . 20
4.4 Coverage against project requirements . . . 20
4.5 Appendix: Manual test cases . . . 21
4.6 Summary . . . 22

5 Evaluation **23**
5.1 Limitations . . . 23
5.2 Future Possibilites . . . 24
5.3 Future possibilities . . . 25

6 Project management **26**
6.1 Project Planning . . . 26
6.2 Team Agreement and Working Practices . . . 27
6.3 Team Roles and Responsibilities . . . 27
6.4 Communication and Collaboration . . . 28
6.5 Challenges Encountered . . . 28
6.6 Lessons Learned . . . 29

Chapter 1

Introduction

Universities are large, complex organisations that provide a wide range of services to their students. At King's College London (KCL), students regularly need support across areas such as academic assessment, welfare, financial matters, careers, and university procedures. While specialist services exist for each of these areas, students frequently direct their queries to generalist staff—such as programme officers and personal tutors—who must then triage, redirect, or respond to a significant volume of incoming messages. This creates a recurring burden: staff spend considerable time handling queries that are routine, misdirected, or duplicated across multiple recipients, while overdue and unanswered queries risk going unnoticed altogether. The absence of a structured query-management process means that both staff efficiency and student experience suffer.

The KCL Ticketing System addresses this problem by providing programme officers with a centralised web application for managing incoming student queries as support tickets. Programme officers gain a single, structured workspace where every query is visible, trackable, and attributable—eliminating the risk of queries going unanswered and removing the duplicated effort that arises when the same query reaches multiple members of staff. Students benefit from a reliable, transparent channel for raising issues, with confidence that their query has been received and will be handled. The system further reduces staff workload by enabling immediate, automated preliminary responses to routine queries, reserving human attention for the cases that genuinely require it.

The system is a full-stack web application accessible through any modern browser. The back end is built with **Django 5** and **Django REST Framework** (Python), exposing a JSON API secured with JWT authentication. The front end is a single-page application built with **React 18** and **Redux Toolkit** (JavaScript). An AI-powered support assistant uses the **Google Gemini API** to provide automated responses to routine student queries. Data is persisted in **SQLite** for development, with the architecture supporting migration to a production-grade relational database.

Chapter 2

Product

Instructions!

Please remove all expectations and assessment environments from this page before submitting, including this instruction. This is placeholder content to help you write the report. It must not be included in the final report.

Length requirements

There are no strict length requirements for this chapter. However, it is important that the writing is clear and concise. Avoid repeating yourself. It is expected that a typical project needs 2–4 pages, but this can vary considerably from project to project.

Expectations

This is a fairly short chapter that outlines what the team has developed and why. It will be used to try and understand the product your team has developed and how the features we can observe provide value to certain people. The chapter should describe what the team has delivered, not what the team hoped to deliver.
The section headers below provide a suggested structure. You should feel free to change it or extend it.

2.1 Objectives, context and key stakeholders

Universities such as King's College London are large, complex organisations that provide a wide range of services to students. Students frequently require support across areas including academic assessment, welfare, financial matters, careers, and general university procedures. While specialist services exist for each of these areas, generalist staff: such as programme officers and personal tutors, often act as the first point of contact. Personal tutors, who are typically teaching staff, must manage these queries alongside their existing teaching, research and administrative duties.

This situation creates several recurring challenges: programme officers and tutors spend substantial time handling misdirected or duplicated queries, while overdue or unanswered queries risk being overlooked. The lack of a structured query-management process reduces

staff efficiency and can negatively impact the student experience. For example, students may experience delayed responses to time-sensitive queries, and staff may encounter duplicated work or increased administrative burden.

2.1.1 Project Objectives

The primary objective of this project is to develop a Ticketing System that provides programme officers with a centralised web application for managing incoming student queries. The core aim is to create a single, structured workspace where every query can be viewed, tracked and is accounted for. The system aims to deliver the following benefits:

1. **Improve staff efficiency** – Reduce time spent on routine or duplicated queries, allowing staff to focus on complex or high-priority issues.
2. **Enhance student experience** – Offer students a reliable and transparent channel for submitting queries, with confirmation of receipt and updates on progress.
3. **Increase accountability and traceability** – Ensure that all queries are logged, assigned and tracked; preventing them from being lost or overlooked.
4. **Support evidence-based decision making** – Enable collection of data on query types, response times and resolution patterns to inform process improvements.
5. **Enable scalability** – Provide a system capable of supporting increasing query volumes.

2.1.2 Key Stakeholders

The main stakeholders of the Ticketing System are:

- **Programme officers and generalist staff** – Require a structured interface to manage queries efficiently, prioritise work and reduce repetitive tasks. They expect a reliable system that supports their workflow.
- **Students** – Seek timely, consistent support. They expect confirmation that their queries have been received and visibility of progress until resolution.
- **University administration** – Interested in improving operational efficiency and student satisfaction. They benefit indirectly from enhanced staff productivity and reporting capabilities that enable monitoring of service performance.

Secondary stakeholders include academic departments and IT support teams responsible for maintaining the system and ensuring smooth integration with existing university services.

2.1.3 Value Proposition

By centralising query management, the Ticketing System reduces duplicated effort, improves response times and provides a transparent communication channel for students. Staff can focus on cases that genuinely require human attention, while the university

gains actionable insights into recurring issues, supporting continuous improvement in student services. The system therefore, enhances efficiency, accountability and transparency; delivering measurable benefits to all stakeholders involved.

Expectations

This section aims to explain why you develop the project.
A good starting point are the project's key stakeholders and, insofar it is relevant, the background or context for the project. Provide enough information for the reader to understand the project objectives.
As you explain the project objectives, focus exclusively on what value the system adds to the stakeholders. Your objectives should justify *why* you are building the system. Do not confuse objectives with requirements: software requirements are *not* objectives. The software requirements explain *what* your system can do. In other words, they are a way to achieve objectives. Software requirements do *not* belong in this section. Indeed, if objectives and requirements are confused, the reader may be left with the impression that the project is a solution in search of a problem.
Your project should have at least one objective. Otherwise, the project is a pointless exercise. Having many objectives does not necessarily make for a better project, and could leave the project unfocussed.

2.2 Requirements and specifications

The requirements of the Ticketing System are derived directly from the project objectives described in Section 2.1. They define the capabilities and constraints of the system needed to meet the needs of students, programme officers and administrators. To present these requirements clearly, this section uses a user story approach, which expresses the system's functionality from the perspective of each stakeholder and links directly to the value the system provides.

Each user story describes a specific goal that a stakeholder wishes to achieve using the system. The corresponding system feature or page illustrates where the functionality can be experienced in the implemented platform. In addition to functional requirements, key non-functional requirements are listed to ensure performance, security, accessibility and scalability of the system.

The following tables summarise both the functional and non-functional requirements, providing a clear map between stakeholder needs, project objectives and system features:

Stakeholder	**User Story / Requirement**	**Objective Supported**	**System Feature / Page**
Student	Submit a support ticket with issue type, description and attachments	Enhance student experience	TicketFormPage
Student	View the status and history of submitted tickets	Increase accountability and transparency	UserDashboardPage
Student	Receive immediate confirmation or automated response upon ticket submission	Enhance student experience, improve staff efficiency	TicketFormPage, Automated Email / AI Chatbot
Student	Request meetings with staff, specifying date, time and topic	Improve staff efficiency, enhance student experience	MeetingRequestPage
Student	Close a ticket once their issue is resolved	Increase accountability and transparency	UserDashboardPage
Student	Reply to tickets after staff response	Enhance student experience	UserDashboardPage
Student	Receive progress updates on submitted tickets	Enhance student experience, improve transparency	UserDashboardPage
Staff / Programme Officer	View all tickets assigned to them	Improve staff efficiency	StaffDashboardPage
Staff / Programme Officer	Assign tickets to the appropriate staff member	Improve staff efficiency, increase accountability	StaffDashboardPage, TicketsManagement Component
Staff / Programme Officer	Update ticket status and close tickets once resolved	Increase accountability and transparency	StaffDashboardPage
Staff / Programme Officer	Accept or deny student meeting requests based on office hours	Improve staff efficiency	StaffDashboardPage, OfficeHours Management
Staff / Programme Officer	Redirect tickets to other staff members if more suitable	Improve staff efficiency, ensure correct support	StaffDashboardPage, TicketsManagement Component
Administrator	Create, update and deactivate user accounts with roles	Increase accountability and transparency	UsersManagement Component
Administrator	View, update or delete any ticket in the system	Increase accountability and transparency	TicketsManagement Component
Administrator	View average reply time per department	Support evidence-based decision making	Statistics Component / AdminDashboard
Administrator / Staff	Report inappropriate tickets	Maintain integrity of support	AdminDashboard/ StaffDashboardPage

Table 2.1: User story requirements mapped to system objectives and implemented features

ID	Non-Functional Requirement
NFR-1	The system shall be accessible via modern web browsers on both desktop and mobile devices.
NFR-2	The system shall authenticate users securely using JWT tokens.
NFR-3	Ticket submission and updates shall respond within 2 seconds under normal load.
NFR-4	The system shall maintain data integrity, ensuring that all tickets, replies, and meeting requests are reliably stored.

Table 2.2: Non-Functional Requirements

Expectations

Use this section to provide the requirements and/or specifications for the project. There are not strict rules on the format you should be using. You can employ user stories or more conventional descriptions of requirements/specifications. However, you should be consistent. Explain how the requirements/specifications derive from the objectives. If it might not be clear to the reader, identify where the reader can experience the implementation in the system that was delivered.

Chapter 3

Design and implementation

3.1 System Architecture

This section describes the overall architecture of the system and how the frontend, backend, and database interact to support the ticketing platform.

3.1.1 System Layers

The PackItUp ticketing system follows a three-tier client–server architecture consisting of a React frontend, a Django REST backend, and a relational database. This layered design separates presentation logic, application logic, and data storage.

The frontend is implemented as a React single-page application (SPA) running on port 3000, responsible for rendering the user interface, managing routing, and maintaining application state. The frontend communicates with the backend through RESTful API calls secured with JSON Web Token (JWT) authentication.

The backend is implemented using Django and Django REST Framework, running on port 8000, and provides the system's business logic including ticket management, meeting scheduling, and user authentication.

While an SQLite database is used for persistent storage during local development, the deployed production environment utilizes a cloud-hosted PostgreSQL database provided by Neon (neon.tech). Access to both is managed seamlessly through Django's Object Relational Mapping (ORM) layer, ensuring environment parity.

[System Architecture Diagram]

3.2 Component Design

This section describes the structure of the frontend components and how they organise the user interface and system functionality.

3.2.1 Frontend Pages

The frontend application is implemented as a React single-page application and is organised into modular page components located within the src/pages/ directory. Each page represents a major functional area of the system and interacts with the backend through REST API requests. For example, the UserDashboardPage allows students to view and manage their submitted tickets, while the StaffDashboardPage enables staff members to review and update assigned tickets. Ticket creation is handled through pages such as TicketFormPage, while additional pages provide supporting functionality including browsing staff members, requesting meetings, and viewing frequently asked questions.

3.2.2 Administrative Pages

Administrative functionality is implemented through specialised components located within the src/components/Admin/ directory. These components provide interfaces for managing system data and monitoring platform activity. The AdminDashboard acts as the central navigation hub for administrative operations and provides an overview of system statistics. Components such as TicketsManagement allow administrators to perform CRUD operations on tickets, including assigning staff members and updating ticket status. The UsersManagement component enables the management of user accounts and roles, while the Statistics component provides analytics and reporting capabilities, including the ability to export system data.

3.2.3 Shared Components

The application also includes shared components that provide reusable interface elements across multiple pages. These components ensure consistent navigation and layout throughout the system, for example the UserNavbar, which provides navigation for authenticated users, and components supporting the FAQ interface.

3.2.4 State Management and API Communication

Global application state is managed using Redux Toolkit, which provides a centralised store for managing shared data across the application. The authSlice manages user authentication state, while the adminSlice maintains state related to administrative dashboard data. Communication between the frontend and backend is handled through REST API calls using a centralised API utility (apiFetch). This utility automatically includes authentication credentials in requests by attaching a JSON Web Token to the Authorization header in the format Bearer <JWT_TOKEN>. Axios is also used for certain operations requiring additional request configuration.

[Frontend component diagram]

3.3 Backend Structure

This section describes the organisation of the backend application and the responsibilities of its main modules.

3.3.1 Django Applications

The backend is implemented using Django and Django REST Framework and is structured into several Django applications responsible for different areas of system functionality. The primary application, KCLTicketingSystems, contains the core logic of the ticketing platform, including ticket creation and management, user management, meeting request handling, and office hours management. An additional application, AIChatbot, provides an AI-powered support assistant using the Gemini API. This module offers a chatbot interface that allows users to interact with an automated support system through a Django-rendered chat page and REST API endpoints.

3.3.2 View Organisation

Backend functionality is implemented through Django REST views located within the KCLTicketingSystems/views/ directory. These views expose API endpoints responsible for key operations such as ticket creation, ticket updates, replies, staff dashboards, and meeting requests. Administrative functionality is handled through separate views that provide system management capabilities including user administration, ticket oversight, and reporting.

3.3.3 Permissions and Access Control

Access control within the backend is implemented using a role-based permissions system. Users are assigned roles such as student, staff member, or administrator, which determine the actions they are permitted to perform within the system. Most API endpoints require authentication through JSON Web Tokens, ensuring that only authorised users can access protected resources. Custom permission classes are implemented to restrict certain operations to staff or administrators, such as managing meeting requests or performing administrative actions.

3.3.4 AI Helper Development with Gemini API

An AI-powered support assistant was developed to enhance the student experience by providing immediate, automated responses to routine queries. This feature acts as a first-line support mechanism, potentially resolving issues before a formal ticket is created. The assistant is implemented using the Google Gemini API, integrated through a dedicated backend service to provide high-performance natural language processing.

From an architectural perspective, the AI helper follows a secure request-response pipeline.

Backend Proxying: The student submits a query through the React frontend to a protected Django endpoint, and the backend forwards the request to the Gemini API. This proxy-based design is a key security control because API credentials remain server-side and are never exposed in client-side code.

Prompt Engineering and Contextual Grounding: To keep responses relevant to King's College London (KCL) procedures, the system uses prompt engineering and contextual grounding. Queries are enriched with system-specific context, including academic assessment types, welfare services, and financial support workflows, to reduce hallucinations and improve policy-aligned, actionable output.

Stateless Inference: The integration uses Gemini's generative capabilities to interpret user intent and return sanitised JSON responses to the frontend. This supports a responsive, dashboard-integrated chat experience without requiring persistent model-side session state.

Operational safeguards were also implemented to maintain service quality.

Input Validation and Sanitization: User inputs are validated and sanitised at the Django layer before forwarding requests to Gemini, reducing prompt-injection risk and improving data integrity.

Graceful Error Handling: The backend includes exception handling for API failures such as rate-limit events and timeout errors. If a reliable answer cannot be generated, the interface guides the student to the TicketFormPage for manual support submission.

Optimization: Generation parameters, including temperature and maximum token limits, were calibrated to balance response depth with low-latency interactions in the student dashboard.

[Backend component diagram]

3.4 Database Design

[UML Class Diagram]

This section describes the structure of the system's persistent data storage and the relationships between the main entities used by the application.

The database schema is implemented using Django ORM models and is designed to support ticket management, communication between users and staff, and meeting scheduling. The central entity within the system is the User model, which extends Django's AbstractUser class to include additional attributes such as a KCL student identifier (k_number), department information, and a role field used to distinguish between student, staff, and administrator accounts.

Support requests are represented by the Ticket model, which stores information including the issue type, description, priority level, and current status of the ticket. Each ticket is associated with the user who created it and may be assigned to a staff member responsible for resolving the issue. Tickets may contain multiple replies and attachments, enabling threaded communication between users and staff members.

Replies are represented by the Reply model, which links users and tickets and allows users to respond to existing discussions. The model includes a self-referential relationship that supports nested replies, enabling threaded conversations within a ticket.

Meeting scheduling functionality is implemented through the MeetingRequest model. This entity stores meeting requests submitted by students to staff members and records details such as the requested meeting date and time, a description of the meeting topic, and the current request status. Staff availability is defined through the OfficeHours model, which records weekly time slots during which staff members are available to meet with students.

Attachments associated with support tickets are represented by the Attachment model, which stores file metadata and links uploaded files to the corresponding ticket.

Several constraints are enforced within the data model to maintain data integrity. User email addresses and student identifiers are required to be unique. Meeting requests are validated to ensure that the requested meeting time falls within the staff member's defined office hours, and office hour records enforce that start times occur before end times.

3.5 API Design

The frontend communicates with the backend through a RESTful API implemented using Django REST Framework. The API exposes endpoints for authentication, ticket management, meeting requests, and administrative operations. Most endpoints require authentication using JSON Web Tokens (JWT), which are included in the Authorization header of requests in the format Bearer <token>. API endpoints are organised according to user roles and system functionality.

3.5.1 Authentication Endpoints

Endpoint	**Method**	**Purpose**
/api/auth/register/	POST	Create a new user account
/api/auth/token/	POST	Obtain JWT access and refresh tokens
/api/auth/token/refresh/	POST	Refresh an expired access token
/api/users/me/	GET	Retrieve the currently authenticated user

Table 3.1: Authentication API endpoints

Authentication tokens are configured with a one-hour lifetime for access tokens and a longer validity period for refresh tokens.

3.5.2 Ticket Management Endpoints

Endpoint	**Method**	**Purpose**
/api/tickets/	POST	Create a new support ticket
/api/dashboard/	GET	Retrieve tickets belonging to the authenticated user
/api/dashboard/tickets/{id}/close/	POST	Close an existing ticket
/api/staff-dashboard/	GET	Retrieve tickets assigned to staff members
/api/staff/dashboard/{id}/update/	PATCH	Update ticket status

Table 3.2: Ticket management API endpoints

These endpoints allow users to create and manage support tickets while enabling staff members to review and update assigned tickets.

3.5.3 Meeting and Staff Endpoints

Endpoint	Method	Purpose
/api/staff/	GET	Retrieve list of staff members
/api/staff/{id}/	GET	Retrieve staff profile and office hours
/api/meeting-requests/	POST	Submit a meeting request
/api/staff/dashboard/meeting-requests/	GET	Staff view incoming meeting requests
/api/staff/dashboard/meeting-requests/{id}/accept/	POST	Accept a meeting request
/api/staff/dashboard/meeting-requests/{id}/deny/	POST	Reject a meeting request

Table 3.3: Meeting request and staff endpoints

Meeting requests are validated against staff office hours to ensure that appointments can only be scheduled during available time slots.

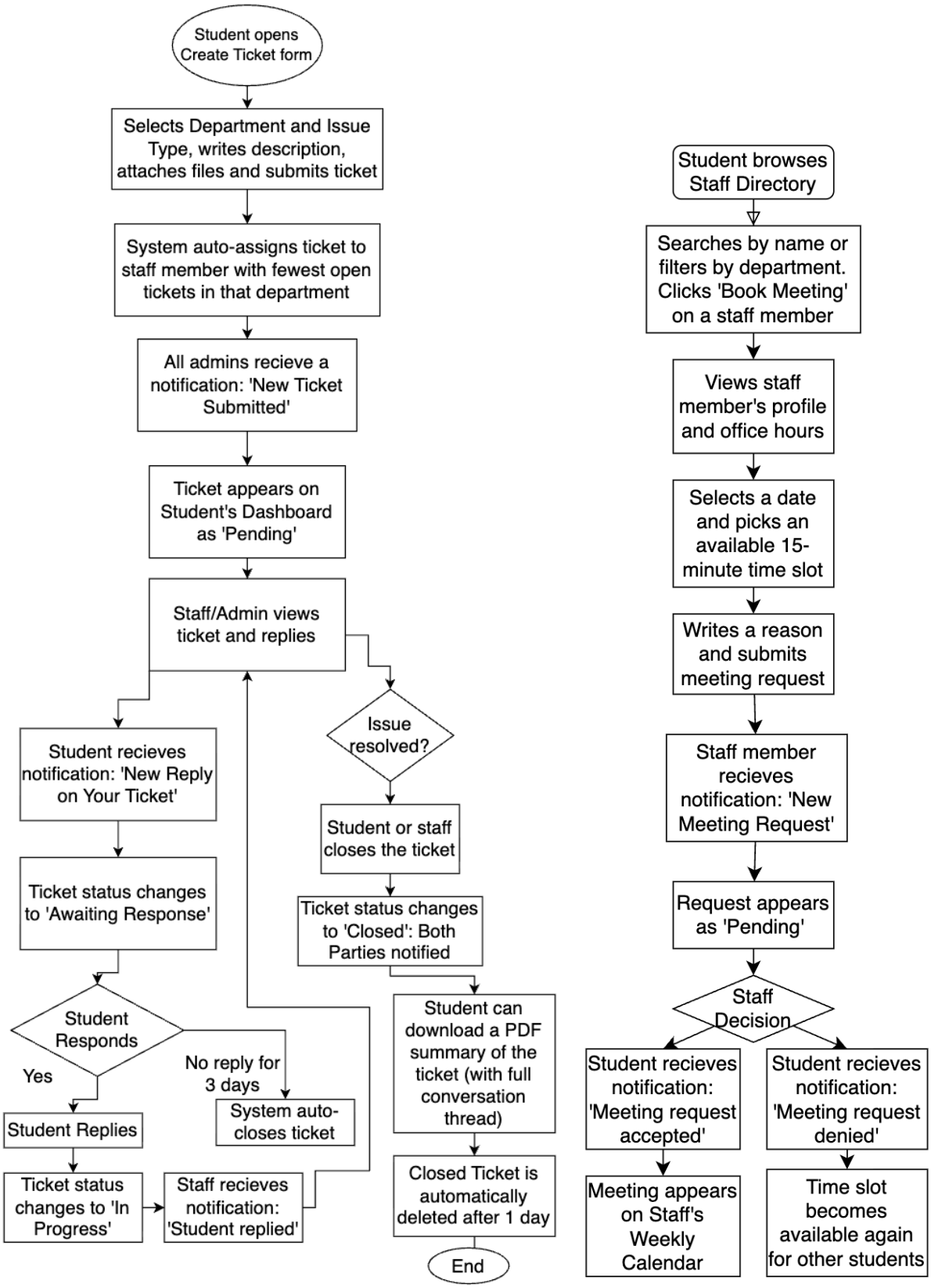
3.5.4 Administrative Endpoints

Endpoint	Method	Purpose
/api/admin/dashboard/stats/	GET	Retrieve system statistics
/api/admin/tickets/	GET	Retrieve all tickets
/api/admin/tickets/{id}/update/	PATCH	Update ticket information
/api/admin/tickets/{id}/delete/	DELETE	Delete a ticket
/api/admin/users/	GET	Retrieve all users

Table 3.4: Administrative API endpoints

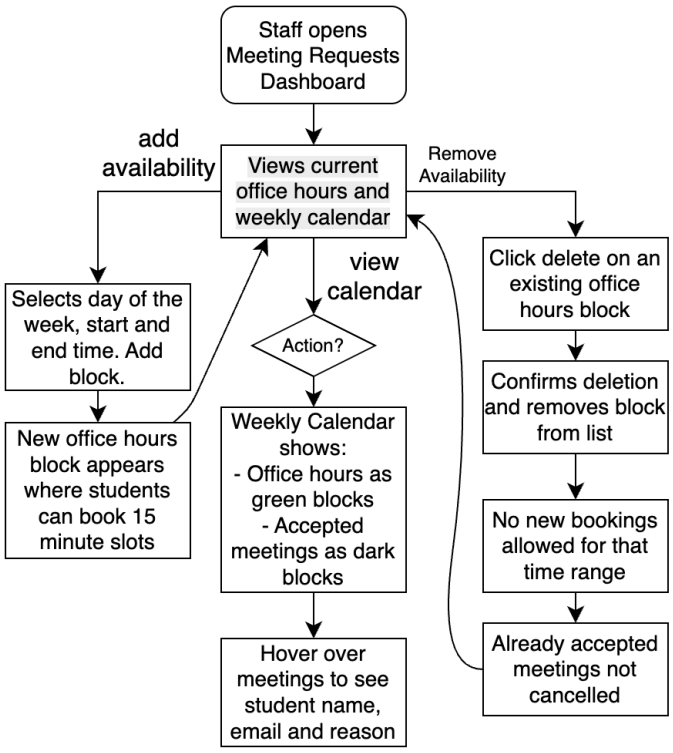
3.6 System Workflows

This section illustrates the main operational processes of the system, such as ticket creation and meeting requests, and how different components interact to complete these tasks.



(a) Ticket Creation Workflow (b) Meeting Request Workflow

Figure 3.1: System workflows (part 1)



(a) Office Hours Workflow

Figure 3.2: System workflows (part 2)

3.7 Design decisions and rationale

This section discusses the principal design and implementation choices, how they appear in the delivered system, and why alternatives were not adopted. Where helpful, choices are related to established software engineering ideas: *separation of concerns*, modular decomposition, risk and cost trade-offs, and maintainability under team development.

3.7.1 Separation of frontend and backend (React and Django REST)

The system was implemented as a React single-page application talking to a Django REST backend. This reflects a deliberate *separation of concerns*: the UI (presentation, routing, client state) is isolated from server-side business rules, persistence, and authorisation. In practice, students and staff interact with page components under `src/pages/` and shared UI under `src/components/`, while tickets, meetings, and roles are enforced in Django views and serializers—a clear boundary that matches the three-tier structure described earlier.

Alternatives considered. A *monolithic* Django application using server-rendered templates would have reduced the number of repositories and avoided CORS, aligning with a classical Model–View–Controller style within one process. It was rejected because rich, interactive dashboards (filtering, real-time feel, client-side validation) are more naturally expressed with a component-based SPA, and the course's learning outcomes favoured a modern split-stack delivery. Lighter Python frameworks (e.g. Flask or FastAPI) were not chosen because Django's built-in user model, admin patterns, and ORM reduced bespoke

code for authentication and data modelling, improving delivery speed for a fixed-term group project.

REST as the integration style. Exposing a JSON REST API is a conventional *client–server* contract: stable resources (`/api/tickets/`, `/api/staff/`, etc.) allow the frontend to evolve independently of URL structure and support future clients (e.g. a mobile app) without rewriting core logic—an application of the *open–closed* idea at the API boundary.

3.7.2 Generative AI: Google Gemini API versus self-hosted inference (e.g. Ollama)

The AI helper is implemented by calling the **Google Gemini API** from the backend (see Section 3.3.4), not by running a large language model on the group's own infrastructure.

Why a managed API was chosen. From a software engineering perspective, outsourcing inference to a managed API reduces *operational risk*: the team does not provision GPUs, tune model serving, or patch inference runtimes. Integration is limited to HTTP requests, authentication, and prompt design—complexity that fits the ticketing domain without becoming an MLOps project. The end product reflects this as a thin server-side adapter (Django endpoint → Gemini) and a predictable failure mode (rate limits, timeouts) handled in application code.

Why Ollama (or similar self-hosted stacks) was not used. Early prototypes considered running a local or server-hosted model via **Ollama**, which would keep data on infrastructure the team controls and avoid per-token cloud pricing. This path was set aside for several reasons. Deploying Ollama for production-quality use implies either (i) sufficient CPU/RAM on the deployment host for acceptable latency, or (ii) GPU-backed cloud instances, which incur *ongoing cost* and operational overhead (monitoring, restarts, model versioning). Coordinating consistent deployment across developers and a hosted environment would also have been harder than calling an API with a server-stored key. For a coursework timeline and team size, the managed API offered lower deployment friction and more predictable integration effort, at the expense of dependency on Google's service and quota limits—trade-offs accepted and documented in the evaluation chapter.

3.7.3 Authentication and data store choices

JWT. Token-based authentication was chosen for stateless API access from the SPA. Alternatives include session cookies with CSRF protection for same-site deployments; JWT was preferred because it keeps authorisation explicit in API requests and matches common SPA tutorials and tooling, though it complicates token revocation without extra infrastructure (see below).

SQLite versus PostgreSQL. Local development uses SQLite to minimise setup; production uses Neon-hosted PostgreSQL. This follows the idea of *environment parity* through the ORM: the same Django models run against both, reducing "works on my machine" defects while keeping developer onboarding lightweight.

3.7.4 How decisions are reflected in the end product

The architecture is not only conceptual: the repository layout mirrors it (React `src/` tree, Django `apps/`, REST routes in tables in Section 3.5), role-based access appears in both UI routes and permission classes, and the AI assistant is a concrete module (`AIChatbot`) rather than an ad hoc script. That alignment between design intent and implementation makes the system easier for future maintainers to navigate.

3.7.5 Benefits of the chosen architecture

Separating the frontend and backend allows parallel development and clearer ownership of defects. React's component model supports reuse of navigation and layout; Django provides ORM migrations, validation, and a consistent pattern for API endpoints. Together, these support maintainability and incremental extension (e.g. new ticket types or admin reports) without a full rewrite.

3.7.6 Limitations and trade-offs

Maintaining two codebases increases coordination and requires CORS and proxy configuration. Stateless JWT access tokens simplify the server but make immediate revocation harder without a token blocklist or short lifetimes paired with refresh flows—acceptable for this project's threat model.

SQLite is unsuitable for high-concurrency production workloads but is acceptable for development; PostgreSQL on Neon addresses deployment needs. The generative AI feature depends on external API availability and quotas, which constrains load testing and demo frequency.

3.7.7 Other alternative approaches considered

A server-rendered Django-only application was rejected for limited richness of interactive dashboards. Next.js, GraphQL, and microservices were judged to add operational and learning-curve cost disproportionate to team size and scope. A REST monolith (single Django service with multiple apps) was preferred over microservices because the domain is a single bounded context (ticketing and related workflows), matching the principle of starting with a *modular monolith* before splitting services.

Chapter 4

Testing

4.1 Testing approach

Our testing strategy combined **automated testing** (unit, integration, and API/UI behaviour tests) with targeted **manual testing** for user experience and end-to-end acceptance checks.

The approach followed three principles:

1. **Risk-first coverage:** prioritise workflows that affect core stakeholder value (ticket creation, dashboard visibility, status updates, replies, meeting requests, and admin controls).
2. **Fast feedback:** run backend and frontend test suites locally via scripted commands before merging.
3. **Defect containment:** use regression tests when bugs are found so the same issue is less likely to recur.

4.2 Automated testing

4.2.1 Suite scope and structure

The delivered codebase contains a broad automated suite across backend and frontend:

- **Backend (Django/DRF):** 39 test files and 594 test cases.
- **Frontend (React/Jest):** 42 test files and 354 test cases.
- **Total:** 81 test files and 948 automated test cases.

Backend tests are concentrated in `KCLTicketingSystems/tests/`, with additional configuration tests in `KCLTicketingSystem/tests/` and chatbot tests in `AIChatbot/tests.py`. Frontend tests are distributed across pages, components, API service modules, utility functions, and Redux slices.

4.2.2 Backend testing (unit and integration)

Backend tests cover:

- **Model-level unit tests:** validation, field constraints, timestamps, relationships, and string representations.
- **API integration tests:** request/response behaviour, authentication (JWT), permission boundaries, role-based access, and error handling.
- **Feature/regression tests:** ticket conversation, rich-text handling, notification behaviour, PDF export, admin reporting, filtering, and staff assignment logic.

```
def test_admin_ticket_update_not_found(self):
    resp = self.client.patch(
        f"/api/admin/tickets/99999/update/",
        {"status": "closed"}
    )
    self.assertEqual(resp.status_code, status.HTTP_404_NOT_FOUND)
    self.assertIn("error", resp.data)
```

Listing 4.1: Backend API integration test example (admin edge-case handling)

Figure 4.1: Code snippet image: backend API test logic

4.2.3 Frontend testing (component and behaviour)

Frontend tests use React Testing Library and Jest to verify:

- **Rendering and interaction:** pages, components, and conditional UI states.
- **Form behaviour:** validation, field updates, submission success/failure paths.
- **Routing and access control:** protected routes and role-based navigation.
- **State management:** Redux slice reducers and async thunk outcomes.
- **API client behaviour:** request headers, payload handling, and error mapping.

```
test("submits ticket and navigates to dashboard on success", async ()
    => {
  apiFetch.mockResolvedValue({ id: 1 });
  render(<CreateTicketPage />);

  await userEvent.selectOptions(screen.getByLabelText(/department/i),
      "Informatics");
  await userEvent.click(screen.getByRole("button", { name: /submit
      ticket/i }));

  await waitFor(() => {
    expect(apiFetch).toHaveBeenCalledWith("/tickets/",
        expect.any(Object));
  });
  expect(mockNavigate).toHaveBeenCalledWith("/dashboard", { replace:
      true });
});
```

Listing 4.2: Frontend behaviour test example (ticket submission flow)

Figure 4.2: Code snippet image: frontend component/integration-style test

4.2.4 Execution pipeline

To keep execution simple for all team members, test orchestration scripts were provided:

- `run_all_tests.bat` (Windows)
- `run_all_tests.sh` (Linux/macOS)

These run backend tests first, then frontend tests, giving a single pass/fail checkpoint.

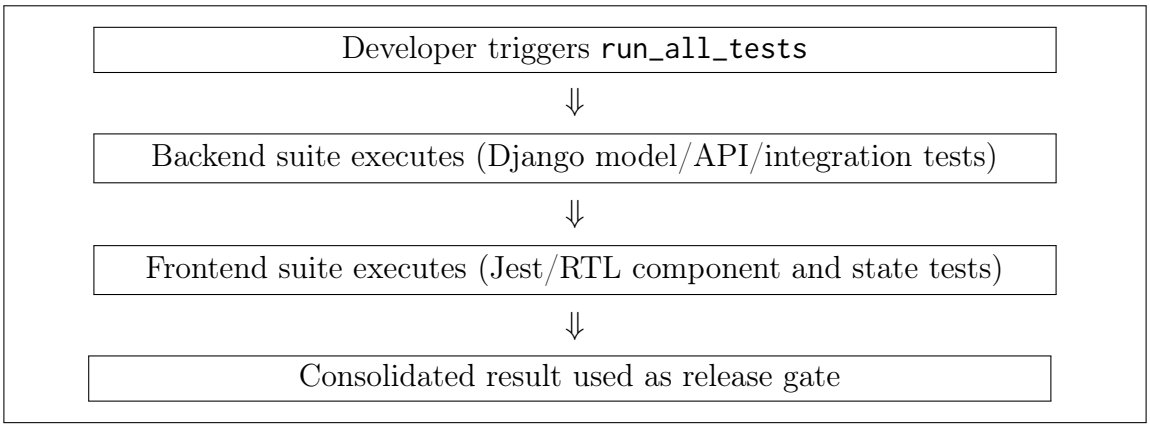


Figure 4.3: Flow diagram: automated test execution path

4.2.5 Current observed run status

On the most recent full backend run in this branch:

- **574 backend tests** were discovered and executed.

- **569 passed**, with **1 failure** and **4 errors** detected.

This is useful evidence that the suite is active and capable of catching regressions. The failing set is concentrated in edge/error-path scenarios, which indicates the test suite is probing beyond only happy paths.

4.3 Manual testing

4.3.1 Purpose of manual testing

Manual testing was used primarily for areas where *human judgement* is essential:

- visual consistency and usability of multi-step user journeys;
- perceived clarity of error/success messaging;
- role-specific UX checks (student vs. staff vs. admin);
- exploratory checks after UI or workflow changes.

4.3.2 Extent of reliance on manual testing

The team relied mainly on automated tests for repeatable correctness and regression control. Manual testing was complementary rather than primary. In practice:

- **Automated tests** handled most deterministic validation (input rules, permissions, response codes, state transitions).
- **Manual tests** were used for end-to-end confidence in user-facing behaviour and presentation quality.

4.3.3 Limits of automated and manual testing

extbfAutomated testing limits:

- may miss usability/accessibility perception issues;
- can become brittle when UI markup changes;
- does not fully represent all browser/device behaviours unless explicitly expanded.

extbfManual testing limits:

- slower and harder to repeat at high frequency;
- vulnerable to inconsistent execution between testers;
- weaker as a long-term regression safety net.

4.4 Coverage against project requirements

Table 4.1 links representative requirements (Chapter 2) to concrete testing evidence.

extbfReq. ID	**Requirement summary**	**Testing evidence (examples)**
FR-Student-Create	Student can submit support tickets	Backend submit endpoint tests (valid/invalid payloads), frontend `CreateTicketPage` validation/submission tests, manual end-to-end form submission checks
FR-Student-Track	Student can view and track tickets	Dashboard/API tests for ticket retrieval, route protection tests, manual dashboard walkthrough checks
FR-Staff-Manage	Staff can review/update/close assigned tickets	Staff dashboard and update endpoint tests, close-ticket tests, permission boundary tests, manual workflow checks
FR-Admin-Control	Admin can manage tickets/users and monitor stats	Admin API tests (CRUD, filters, exports, statistics), auth/role tests, manual admin console verification
NFR-Security	Secure role-restricted access with JWT	Token obtain/refresh tests, unauthorised/forbidden endpoint tests, private route frontend tests
NFR-Integrity	Reliable storage and consistent state transitions	Model tests (tickets, replies, notifications, meeting requests), integration tests for update/reassign/close flows

Table 4.1: Requirement-to-test traceability

4.5 Appendix: Manual test cases

Table 4.2 provides the required manual test log with explicit actions, expected observations, date, and outcome.

extbfID	**Manual test**	**Tester actions**	**Expected observation**	**Last performed**	**Status**
MT-01	Student ticket creation	Login as student; open Create Ticket page; complete department, issue type, details; submit.	Success confirmation appears and ticket is visible in student dashboard with correct metadata.	27/03/2026	Pass
MT-02	Student input validation UX	Leave mandatory fields blank and submit; enter malformed K-number/email patterns where applicable.	User receives clear inline/global validation guidance and submission is blocked until corrected.	27/03/2026	Pass
MT-03	Staff ticket lifecycle	Login as staff; open assigned ticket; add reply; update status; close ticket.	Ticket status updates persist and are reflected in both staff and student views.	27/03/2026	Pass
MT-04	Meeting request handling	Student requests meeting; staff opens requests and accepts/denies.	Request transitions to expected status and is shown correctly in relevant pages.	27/03/2026	Pass
MT-05	Admin ticket management	Login as admin; filter/search tickets; open ticket detail; edit assignment/status/priority.	Filtered results are accurate and updates are persisted and immediately visible.	27/03/2026	Pass
MT-06	Admin user administration	Open user management; edit user role/profile; attempt forbidden self-delete path.	Valid updates are applied, and restricted actions are blocked with clear feedback.	27/03/2026	Pass
MT-07	Notification interaction	Trigger ticket/reply events; open notification bell; mark notification as read.	Notification count and read state update correctly without inconsistent UI behaviour.	27/03/2026	Pass

extbfID	**Manual test**	**Tester actions**	**Expected observation**	**Last performed**	**Status**
MT-08	AI chatbot fallback behaviour	Open chatbot; send normal prompt; simulate unavailable backend/runtime.	Normal response path works; failure path shows user-safe error message without breaking page.	27/03/2026	Pass
MT-09	Access control by role	Attempt direct URL access to protected admin/staff routes using non-authorised account.	User is redirected or blocked according to role and authentication state.	27/03/2026	Pass
MT-10	Cross-browser smoke test	Run core flows in two modern browsers (login, create ticket, dashboard navigation).	No blocking UI regression; forms and routing remain functional in both browsers.	27/03/2026	Pass

Table 4.2: Manual testing appendix: actions, expected results, and outcomes

4.6 Summary

Overall, the project uses a **hybrid testing strategy** with strong automated regression coverage and focused manual validation for usability and end-user confidence. The breadth of the automated suite (948 test cases) provides a solid technical baseline, while manual checks ensure the delivered product remains usable and coherent for students, staff, and administrators.

Chapter 5

Evaluation

The ticketing system developed during this project successfully implements the core functionality required for submitting and managing support requests. Users can create tickets through a structured interface, while the system demonstrates integration with modern web technologies and an AI component designed to assist with handling queries. The implementation meets the primary objectives established to deliver a functional Minimum Viable Product (MVP) within the project timeframe. Despite this, several limitations remain that affect the system's current capabilities and highlight opportunities for further development.

5.1 Limitations

Gemini API credentials and deployment. The AI assistant relies on the Google Gemini API, which requires a secret API key. For security, the key cannot be distributed to every developer machine; as a result, the AI feature is only fully exercised in the deployed environment where credentials are configured securely. Local development therefore does not mirror production behaviour for this component.

API usage limits. The team used a student-oriented Gemini offering with a free usage tier. Daily request and credit limits apply, so sustained load-testing or heavy classroom demos could exhaust the quota. This caps how aggressively the assistant can be used in evaluation and constrains real-world adoption without a paid or institutional plan.

Ticket priority defaults. Support tickets are not assigned fine-grained priorities in the current implementation; a medium default is applied broadly because explicit priority workflows were not essential to the core ticketing and meeting features. This simplifies the model but does not reflect nuanced urgency in a production helpdesk.

FAQ content management. A feature allowing users to add new FAQ entries was not implemented. Besides time constraints, the team judged that an open submission form could clutter the interface and raised unresolved policy questions: whether students, staff, or administrators should author or approve changes. A minimal, read-only FAQ list was preferred for consistency and moderation control.

Email notifications. Automated email (for example via Mailgun or similar APIs) was explored in a prototype but abandoned. Third-party transactional email typically incurs

cost, and the integration proved brittle relative to project time. The final system does not rely on outbound email for ticket updates.

University integrations and realistic staff data. Deeper integration was considered, including Microsoft 365–based flows (for example Teams meeting links) and live staff calendars or working hours from institutional systems. The university could not provide real-time APIs or feeds for staff availability in a form suitable for this project. Designing and securing such integrations would also have added substantial complexity. The team therefore used seeded, representative office hours rather than live directory data, which limits fidelity to real departmental schedules.

Addressing Multiple Tickets. An objective that the team had aimed to achieve was enabling staff to address multiple tickets regarding the same query collectively, rather than handling each ticket individually. This feature would have streamlined responses and reduced duplicate effort. However, due to time constraints and the approaching project deadline, the team chose to prioritise completing the core functionality and ensuring the quality of the implemented features. As a result, this objective was deferred, highlighting an area for improvement in future development.

5.2 Future possibilities

Scalability and infrastructure. The production database is hosted on Neon. The free tier caps storage (for example around 0.5 GB); the project currently uses a small fraction of that (approximately 0.03 GB), so there is headroom for growth. Neon's paid tiers offer higher limits and can support a larger dataset and connection load. The deployed application stack can be scaled horizontally or vertically as traffic grows, and the Gemini integration can be upgraded to higher quotas or enterprise endpoints if AI usage becomes business-critical.

AI-first resolution before ticketing. A natural extension of the current Gemini-powered helper is to deepen its role in the support funnel: students would interact with the assistant first for routine questions, with clear escalation paths only when the model is uncertain or the issue requires human handling. This would reduce duplicate tickets, align with the design goal of first-line automation, and could be combined with analytics on which queries resolve without a ticket. Future work could include stronger grounding in official KCL documentation, explicit confidence thresholds for escalation, and tighter coupling between assistant sessions and ticket creation so context carries over when a formal request is needed.

Chapter 6

Project management

Instructions!

Please remove all expectations and assessment environments from this page before submitting, including this instruction. This is placeholder content to help you write the report. It must not be included in the final report.

Length requirements

There are no strict length requirements for this chapter. However, it is important that the writing is clear and concise. Avoid repeating yourself.

Expectations

This chapter is not assessed directly. Use it as an opportunity to reflect on how the project was managed, what worked well, what did not work so well, and what the team learned from the experience. If there is supporting information you wish to include (e.g. a team charter), add this in the form of appendices.

6.1 Project Planning

At the beginning of the project, the team held an initial planning session to discuss potential project ideas and define the overall goals and scope. Several different system concepts were evaluated based on feasibility, project requirements and the available development timeframe. After discussion, the team conducted a vote to select the most suitable project idea. The final decision was to develop a ticketing system.

Following the selection of the project topic, the team began outlining the structure and functionality of the system. Diagrams and flowcharts were created to visualise the system process and understand how different components of the system would interact. These diagrams assisted in planning the development process and clarifying the workflow. During this stage, the team also identified the Minimum Viable Product (MVP), which included the essential features necessary for the system to operate effectively. Defining the MVP helped prioritise development tasks and ensured that core functionality could be completed within the project deadline.

A project timeline was then established to divide the work into several phases, including planning, development, testing and documentation. This structured approach allowed the team to manage the workload more effectively and maintain steady progress throughout the project.

6.2 Team Agreement and Working Practices

At the start of the project, the team established a formal agreement to define roles, responsibilities, communication methods and expectations. Decisions were made through discussion and majority vote, with polling used if consensus was not possible. Each member was responsible for writing readable, maintainable code and ensuring their commits did not disrupt the repository. The team resolved merge conflicts collaboratively and conducted code reviews together.

The team established clear communication guidelines and used scheduled meetings, WhatsApp and Trello to coordinate, track tasks and monitor progress. Procedures were in place for missed deadlines, work that did not meet standards and temporary unavailability due to illness or other commitments. Conflicts were addressed within the team; it was agreed that lecturers/TAs will be involved only if necessary. The agreement emphasised respect, accountability and a positive team environment, with quality prioritised over quantity in feature development.

6.3 Team Roles and Responsibilities

Responsibilities within the team were distributed based on individual strengths and interests. Hafsa acted as the team leader and chaired all scheduled meetings, ensuring that the project stayed on track. Amey managed the repository and DevOps tasks, including maintaining the version control system and overseeing code integration. Jasmeen took responsibility for recording meeting minutes, documenting decisions and maintaining records of progress. All team members collectively managed the Trello board, updating tasks, monitoring deadlines and coordinating work throughout the project.

Although specific roles were assigned, the project remained highly collaborative. Team members regularly supported one another when challenges arose and responsibilities were occasionally adjusted to ensure that progress continued smoothly. Collaborative tasks such as code reviews and system testing were shared among members to maintain quality and coherence across the project.

Further into the development process, closer to the project deadline, the team adapted by splitting into smaller subgroups to manage core tasks more efficiently. Each subgroup focused on specific areas to ensure all critical tasks were completed on time. The allocation of responsibilities is summarised below:

This structure allowed the team to tackle multiple critical tasks simultaneously while maintaining collaboration and flexibility. By splitting into subgroups, each area of the project received focused attention, which helped ensure the project was completed on time and met the quality standards set by the team.

Table 6.1: Subgroup Task Allocation Near Project Deadline

Subgroup Task	**Team Members**
Testing	Hafsa, Tejas, Benjamin
UI Fixes	Nitin, Ved
Deployment	Amey, Tejas
Report	Jasmeen, Khushi, Nitin
Refactoring	Amey, Nitin

6.4 Communication and Collaboration

Communication played a critical role in the success of the project. The team held scheduled meetings twice a week (every Monday and Thursday) to review progress, discuss completed tasks and to plan ahead for the next stage in development. Between meetings, informal communication was maintained via a WhatsApp group chat to share updates, ask questions and to resolve minor issues promptly. To organise and track tasks, the team used Trello, which allowed for the clear assignment of responsibilities, monitoring of progress and to maintain a record of completed tasks. Code reviews were conducted regularly to maintain high standards of quality. Automated testing was implemented strategically to check functionality without affecting existing code.

6.5 Challenges Encountered

Several challenges arose during the project, which required careful planning and collaboration to overcome. One significant challenge was coordinating meetings, as team members had differing schedules. To address this, a hybrid approach was adopted: those who could attend in person participated in face-to-face meetings, while those who could not joined via WhatsApp calls.

Another challenge was the use of React, as some team members were not familiar with the framework, requiring additional learning time and peer support. The team also encountered difficulties with overly ambitious ideas, which needed to be scaled down to ensure the Minimum Viable Product could be delivered on time.

Additional challenges included merging code and managing version control conflicts in Git, which occasionally slowed progress. Testing and integrating multiple components required more time than initially anticipated, particularly for complex features. Furthermore, the team had to navigate time pressure as deadlines approached, while maintaining quality standards. Clear communication and collaboration were critical in resolving any misunderstandings or minor conflicts that arose during the development process.

To overcome these challenges, the team refined the project scope, prioritised essential features and allocated tasks according to individual strengths. Less critical features were postponed or simplified, ensuring that the core system was completed successfully within the deadline. The hybrid meeting strategy, combined with collaborative support and task management, enabled the team to stay coordinated and productive despite these difficulties. The core aspect which supported the team when navigating these challenges was maintaining flexible communication channels.

6.6 Lessons Learned

Throughout this project, the team gained valuable experience in collaboration and adapting to unexpected challenges.

One of the key lessons was the importance of flexible communication; hybrid meetings, combining in-person attendance with WhatsApp calls, proved essential for accommodating differing schedules and ensuring everyone remained involved.

The team also learned the value of clearly defining a Minimum Viable Product (MVP) and prioritising tasks, which helped keep the project manageable and ensured that critical features were delivered on time.

The development process highlighted the importance of maintaining a structured workflow and clear responsibilities, while also remaining adaptable when adjustments were necessary.

Working closely together on the codebase and coordinating development tasks allowed the team to improve both technical skills and problem-solving abilities, particularly when learning new tools such as React.

The project also reinforced important lessons about managing ambition (balancing the desire to implement additional features with the need to meet deadlines) and the value of maintaining a positive, supportive team environment under time pressure. Overall, these experiences provided insight into effective project management, communication, and teamwork, which will guide the team in future projects and professional settings.